

ANVIL P-04 PCAM PRECISION AGENT - ARCHITECTURE

Submission component: `adapters/theVioneers.py`, class `Engine`.

Problem setting

The PCAM model is frozen. The submission is not allowed to retrain or modify the base memory dynamics. For every corrupted query q , the agent returns a positive diagonal precision vector π . The benchmark then clips and mean-normalizes π before running the PCAM dynamics. Retrieval is scored by improvement over the identity-precision baseline, and anisotropy is scored by spread reduction in the local Hessian geometry.

High-level design

The submitted agent is a gated precision controller with two complementary branches:

1. Retrieval branch for corrupted or ambiguous queries.
2. Geometry branch for clean, high-confidence attractor-like queries.

The key idea is to use precision meaningfully without changing the PCAM model. Corrupted inputs should not trust every PCA dimension equally. Clean inputs, however, can use local Hessian information to modestly improve conditioning around the attractor.

Data flow

`stored_patterns X + model_params` -> Engine initialization

`corrupted_query q` -> similarity/margin analysis -> retrieval precision or geometry

precision -> π vector -> PCAM harness dynamics -> nearest-pattern retrieval

The code is deterministic, NumPy-only, CPU-only, and does not contain seed-specific lookup tables or external trained weights.

Initialization

The Engine receives `stored_patterns` with shape (K, N) and `model_params` containing R, eta, beta, integration parameters, and precision bounds. It stores X, constructs a frozen PCAMModel helper using the provided parameters, and precomputes one geometry template per stored pattern.

Retrieval branch

For a query q, the agent normalizes q and computes cosine similarities $X @ q$. The best match, confidence, and margin to the runner-up are used to estimate how reliable the query is. The retrieval precision uses an adaptive inverse-|q| rule:

- low confidence -> stronger precision shaping
- high confidence -> softer precision shaping
- output is clipped/normalized internally before the harness applies its own projection

This branch is responsible for the strong retrieval delta across public and shifted seeds.

Geometry branch

For each stored pattern, the agent finds the true PCAM equilibrium using `model.find_equilibrium(pattern)`, computes the Hessian at that equilibrium, and optimizes a diagonal preconditioner to reduce the condition spread of $\sqrt{\pi} H \sqrt{\pi}$. This follows the PCAM paper's local-geometry motivation while respecting the benchmark's diagonal-positive precision constraint.

Gating rule

The agent uses geometry directly only when the query is high-confidence and separated from the runner-up. Borderline inputs blend gently. Ambiguous inputs stay retrieval-first to avoid hurting accuracy.

PAGE 3 - RESULTS, REPRODUCIBILITY, AND LIMITATIONS

How to run

From bench-p04-pcam:

```
python self_check.py --adapter adapters.theVioneers:Engine --quick
python run.py --adapter adapters.theVioneers:Engine --seeds 7 13 31 97 211 503 1009
--out report.json
```

Official generated report

The required 7-seed command produced report.json with:

```
mean delta accuracy: +0.1935
worst-seed delta accuracy: +0.1107
mean spread reduction: 1.29x
retrieval score: 70.00 / 70
anisotropy score: 3.21 / 20
total automated score: 73.21 / 90
```

Transfer evidence

Additional summaries are stored in reports.json:

```
PCA-MNIST delta: +0.1500
M2NIST delta: +0.1444
Fashion-MNIST delta: +0.0850
SVHN PCA-64 delta: +0.0533
```

Code quality

The implementation is contained in adapters/theVioneers.py. It uses only NumPy and the provided benchmark files. The README documents setup, approach, commands, dependencies, and verified results. py_compile and JSON validation pass.

Known limitation

The retrieval branch is strong. The anisotropy branch improves spread only modestly, around 1.3x, so it receives limited anisotropy credit. This is documented rather than hidden.